

Introducción a la Programación I

Clase 6: Bucles (for y while)

¿Por qué necesitamos bucles?

Supongamos que tenemos una lista de magos y queremos imprimir el nombre de cada uno:

```
magos = ["alice", "david", "carolina"]  
print(magos[0])  
print(magos[1])  
print(magos[2])
```

Esto funciona... pero, **¿qué pasa si la lista cambia de tamaño?**

Si alguien agrega o elimina un mago, el código se rompe. 🤖

La solución: el bucle **for**

El bucle **for** recorre cada elemento de una lista **automáticamente**, sin importar cuántos elementos haya:

```
magos = ["alice", "david", "carolina"]  
for mago in magos:  
    print(mago)
```

```
alice  
david  
carolina
```

✓ Si la lista crece o achica, el código sigue funcionando.

Anatomía de un **for**

```
for mago in magos:  
    print(mago)
```

- **for** → inicio del bucle
- **mago** → variable temporal (toma el valor de cada elemento)
- **in magos** → la lista que se recorre
- **:** → dos puntos obligatorios
- **Indentación** → todo lo que esté indentado es parte del bucle

⚠ La **indentación** es fundamental en Python. Sin ella, el código falla.

Indentación: lo más importante

```
magos = ["alice", "david", "carolina"]
for mago in magos:
    print(f"El siguiente mago es {mago}.")    # ← dentro del bucle
    print(f"¡No puedo esperar a ver a {mago}!\n") # ← también dentro

print("¡Gracias a todos los magos, fue un gran show!") # ← fuera del bucle
```

```
El siguiente mago es alice.
¡No puedo esperar a ver a alice!
```

```
El siguiente mago es david.
¡No puedo esperar a ver a david!
```

```
...
¡Gracias a todos los magos, fue un gran show!
```

Múltiples líneas dentro del **for**

Todo lo que esté **indentado** se ejecuta en cada iteración:

```
frutas = ["manzana", "banana", "naranja"]
for fruta in frutas:
    fruta_mayus = fruta.upper()
    print(f"Me gusta la {fruta_mayus}")
    print(f"    ({fruta} tiene {len(fruta)} letras)")

print("¡Listo!")
```

```
Me gusta la MANZANA
    (manzana tiene 7 letras)
Me gusta la BANANA
    (banana tiene 6 letras)
...
¡Listo!
```

`range()` — generar secuencias numéricas

`range()` genera una secuencia de números sin crear una lista:

```
# range(stop) → del 0 al stop-1
for i in range(5):
    print(i)    # 0, 1, 2, 3, 4

# range(start, stop)
for i in range(1, 6):
    print(i)    # 1, 2, 3, 4, 5

# range(start, stop, step)
for i in range(0, 10, 2):
    print(i)    # 0, 2, 4, 6, 8
```

`range()` con paso negativo (orden inverso)

```
# Contar hacia atrás
for i in range(5, 0, -1):
    print(i)
```

```
5
4
3
2
1
```

```
# Imprimir una lista al revés con índices
colores = ["rojo", "verde", "azul"]
for i in range(len(colores) - 1, -1, -1):
    print(colores[i])
```


`range()` combinado con listas

Usar `range(len(lista))` para recorrer por índice:

```
frutas = ["manzana", "banana", "naranja"]  
for i in range(len(frutas)):  
    print(f"Posición {i}: {frutas[i]}")
```

```
Posición 0: manzana  
Posición 1: banana  
Posición 2: naranja
```

Útil cuando necesitás el **índice** y el **valor** al mismo tiempo.

`enumerate()` — índice y valor juntos

`enumerate()` es la forma más limpia de obtener índice y valor:

```
frutas = ["manzana", "banana", "naranja"]  
for indice, fruta in enumerate(frutas):  
    print(f"{indice}: {fruta}")
```

```
0: manzana  
1: banana  
2: naranja
```

También podés indicar desde qué número empieza:

```
for indice, fruta in enumerate(frutas, start=1):  
    print(f"{indice}. {fruta}")  
# 1. manzana, 2. banana, 3. naranja
```

Recorrer una lista de listas

Cuando cada elemento de la lista **también es una lista**:

```
puntos = [[1, 2], [3, 4], [5, 6]]
for punto in puntos:
    print(punto)
```

Con **for anidado** para acceder a cada elemento interno:

```
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print() # salto de línea al terminar cada fila
```

```
1 2 3
4 5 6
7 8 9
```

El bucle `while`

El `while` ejecuta un bloque **mientras una condición sea verdadera**:

```
while condicion:  
    # código que se repite
```

- Si la condición es `True` al principio → entra al bucle
- Después de cada iteración → vuelve a evaluar la condición
- Cuando la condición es `False` → sale del bucle

⚠ Si la condición **nunca se vuelve False**, el bucle corre para siempre (bucle infinito).

while — ejemplo simple

Contar del 1 al 5:

```
numero = 1
while numero <= 5:
    print(numero)
    numero += 1    # ← importante: actualizar la variable
```

```
1
2
3
4
5
```

El bucle se detiene cuando `numero` llega a 6 (ya no es `<= 5`).

while vs if — diferencia fundamental

```
# if → evalúa UNA sola vez
edad = 20
if edad >= 18:
    print("Mayor de edad")    # se imprime una vez y listo

# while → evalúa UNA Y OTRA VEZ hasta que sea False
numero = 1
while numero <= 3:
    print(numero)    # se imprime 3 veces
    numero += 1
```

if	while
Decisión (sí/no)	Repetición (mientras...)
Se ejecuta 0 o 1 vez	Se ejecuta 0 o N veces

while — ejemplo con input del usuario

```
mensaje = ""
while mensaje != "salir":
    mensaje = input("Escribí algo (o 'salir' para terminar): ")
    if mensaje != "salir":
        print(f"Dijiste: {mensaje}")

print("¡Hasta luego!")
```

```
Escribí algo (o 'salir' para terminar): hola
Dijiste: hola
Escribí algo (o 'salir' para terminar): Python
Dijiste: Python
Escribí algo (o 'salir' para terminar): salir
¡Hasta luego!
```

Salir de un bucle: **break**

break termina el bucle **inmediatamente**, sin importar la condición:

```
numeros = [1, 3, 7, 4, 2, 9]
for numero in numeros:
    if numero % 2 == 0:
        print(f"Primer número par encontrado: {numero}")
        break    # ← salimos al encontrar el primero
```

Primer número par encontrado: 4

Sin **break**, el bucle hubiera seguido hasta el final de la lista.

break en un while

```
while True:    # bucle "infinito"
    respuesta = input("¿Cuánto es 5 + 3? ")
    if respuesta == "8":
        print("¡Correcto! 🎉")
        break
    else:
        print("Incorrecto, intentá de nuevo.")
```

```
¿Cuánto es 5 + 3? 7
Incorrecto, intentá de nuevo.
¿Cuánto es 5 + 3? 8
¡Correcto! 🎉
```

El patrón `while True` + `break` es muy común en Python.

Resumen: **for** vs **while**

	for	while
Uso típico	Recorrer colecciones	Repetir hasta condición
Cantidad de iteraciones	Conocida de antemano	Puede no conocerse
Riesgo	Bajo	Bucle infinito
Con listas	✓ Ideal	Posible, pero más verbose

```
# for → cuando sabes cuántas veces iterar
for i in range(10):
    print(i)
```

```
# while → cuando no sabes cuántas veces
while not encontrado:
    buscar()
```

Conceptos clave de la clase

- **for** : recorre cada elemento de una lista u objeto iterable
- **Indentación**: define qué líneas pertenecen al bucle
- **range()** : genera secuencias numéricas (con inicio, fin y paso)
- **enumerate()** : obtiene índice y valor al mismo tiempo
- **For anidado**: para recorrer listas de listas
- **while** : repite mientras la condición sea **True**
- **break** : sale del bucle inmediatamente
- Diferencia clave: **if** decide, **while** repite